```
          this.children[s[0]] ←
            new RTEdge(nextEdge.destination,
edge.label+nextEdge.label) #12
      return (deleted,
false)                                                    #13
    else
      return (false,
false)                                                      #16
```

#1 Method `remove` takes a node and the string to delete from the sub-tree rooted at `node`. It returns a couple of Booleans: the first one tells the caller if the key has been successfully deleted, the second one is true if the last link followed became a dangling empty branch and should be pruned.

#2 Check if the string searched is empty: if it is, we traversed the whole path till the final node for the string to delete.

#3 We make sure that current node is marked as an intermediate node.

#4 Search is over, so return to the caller, reporting if the operation was successful (we had saved the value for `deleted` a couple of lines above) and if this node is to be pruned (in case it's a leaf).

#5 Since `s` is not the empty string, we can see if there is an edge matching it, even partially.

#6 Match case #1 (wrt figure 6.15): there is an outgoing edge in `node` whose label is a prefix of the search string, so we can traverse it.

#7 Save in a temporary variable the node at the end of the edge.